



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutierrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Bryant Maximiliano Dzul Perez

Introducción

El desarrollo de software a gran escala requiere de una metodología estructurada que garantice la viabilidad, calidad y escalabilidad del producto final. El Ciclo de Vida del Desarrollo de Software (SDLC, por sus siglas en inglés) proporciona este marco de trabajo esencial, transformando el proceso de creación tecnológica en un esfuerzo de ingeniería organizado y predecible. En el presente documento, se analiza a detalle la aplicación práctica de las etapas del SDLC tomando como caso de estudio una plataforma de alta complejidad y demanda global: Spotify. A través de este análisis, se desglosarán aspectos críticos como la planificación de recursos, la definición de requerimientos funcionales y no funcionales, el diseño de la arquitectura asíncrona, y las fases de pruebas, despliegue y mantenimiento continuo. El objetivo principal es evidenciar cómo la correcta ejecución de cada etapa metodológica es indispensable para construir, sostener y evolucionar sistemas modernos que conectan a millones de usuarios concurrentes de manera ininterrumpida y segura.

Descripción del Sistema:

Nombre del sistema: Spotify

Qué problema resuelve: Facilidad de acceso a un catálogo de música desde cualquier dispositivo, creación de listas de reproducción personal que contenga música del gusto del usuario y un canal de monetización para los creadores de contenido.

Tipo de usuarios:

Oyentes: Usuarios que pagan una suscripción para escuchar música y crear listas.

Artistas: Usuarios que suben contenido a la plataforma.

Administradores: Gestiona usuarios y contenido de la plataforma.

Funcionalidades principales: Reproducción de música, gestión de listas de reproducción, descargas offline de música, pasarelas de pago para suscripciones

Importancia del sistema: Permite conectar artistas con oyentes a nivel mundial mediante una aplicación de suscripción.

Etapas de planeación:

Objetivos: Desarrollar un programa multiplataforma que soporta a miles de usuarios concurrentes y descargas masivas.

Alcance: El proyecto abarca el desarrollo de un aplicación que permite conectar música de artistas de todo el mundo con oyentes de todo el mundo, pasarelas de pago y arquitectura de servidores para el almacenamiento de los audios.

Recursos necesarios: Programadores, servidores en la nube, licencias y acuerdos legales con los artistas y distribuidores.

Riesgos principales: Caídas del servidor por alta demanda, cuellos de botella en la base de datos, sobre costos por ancho de banda de salida en la nube, fugas de seguridad en los datos del pago

Tiempo estimado: Tiempo de desarrollo estimado de 6 meses para el desarrollo de un prototipo funcional, y cerca 1 a 2 años para el programa completo y la adquisición de licencias con discográficas.

Costos aproximados:

Calculo listo. Total bruto: \$437,596.85

RESUMEN DE PARAMETROS

Tiempo estimado	26 semanas (1040 horas totales)
Tarifa profesional	\$350.00 / hora
Costo base mano de obra	\$364,000.00

DESGLOSE POR RUBROS

Concepto	Porcentaje	Monto
Analisis	9.38%	\$34,143.20
Diseno UX/UI	9.38%	\$34,143.20
Complejidad	18.75%	\$68,250.00
Desarrollo	35.16%	\$127,982.40
QA y Testing	14.06%	\$51,178.40
Gestion de proyecto	9.38%	\$34,143.20
Despliegue (DevOps)	3.91%	\$14,232.40

COSTOS ADICIONALES Y FIJOS

Descripcion	Monto
Contingencia (15%)	\$57,077.85
Infraestructura	\$15,000.00
Integraciones	\$1,200.00
Plataforma	\$319.00

Total presupuestado	\$437,596.85
---------------------	--------------

Análisis de Requerimientos:

Requerimientos Funcionales:

RF-01: El sistema debe contar con un método de autenticación con cierre de sesión cada 24 hrs.

RF-02: La plataforma debe permitir a los artistas la carga de archivos en diferentes formatos de audio (WAV, MP3, FLAC, etc)

RF-03: Las búsquedas deben indexar resultados combinados, es decir, artistas, álbumes, canciones y playlist.

RF-04: El sistema debe registrar como una reproducción si el oyente reproduce por más de 30 segundos.

RF-05: La plataforma debe contar con un panel de métricas para el artista, con filtros para canciones o álbumes, y vistas para ordenar por mayor recaudación y mayor reproducción.

RF-06: Los oyentes deben poder reportar una canción, artista o álbum para notificar al administrador y éste se encargue de moderar el contenido desde su panel de administración

Requerimientos no Funcionales:

RNF-01: El tiempo de respuesta del reproductor debe ser menor a 200 milisegundos.

RNF-02: La aplicación debe ser responsive para adaptarse a diferentes dispositivos.

RNF-03: La arquitectura de la base de datos debe soportar miles de usuarios simultáneos.

RNF-04: El sistema debe soportar descargas masivas de música.

RNF-05: La reproducción de música debe estar habilitada para reproducirse en segundo plano.

Arquitectura del Sistema:

Se utilizará una arquitectura MVC con vistas asíncronas para permitir la reproducción de música mientras el usuario navega por la aplicación sin que las canciones se detengan.

Una combinación de Laravel para el backend y React para el frontend durante el desarrollo de la aplicación, y el uso de Docker para asegurar la función del sistema en diferentes dispositivos al momento del desarrollo de la aplicación.

PostgreSQL para la velocidad de consultas pesadas al albergar miles de canciones, artistas y playlist dentro de la base de datos para asegurar consultas rápidas. Además del uso de AWS S3 para guardar las canciones en el servidor.

Desarrollo:

Durante el desarrollo se utilizará tableros con las tareas pendientes para el equipo de desarrollo, el cual contará con su líder, diseñador, desarrollador frontend, desarrollador backend, diseñador de bases de datos y tester

A través de GitHub se tendrá un repositorio del proyecto para control de versiones con diferentes ramas para implementación, main, dev y funciones. Cada rama con un propósito en específico y con sus respectivos pull request.

Los desarrolladores utilizarán su IDE de preferencia pero siempre asegurándose de trabajar en su rama correspondiente para un mejor control de las versiones del sistema.

Pruebas:

Se realizarán pruebas por cada nueva función añadida en búsqueda de errores y para validar que los métodos de los controladores funcionan correctamente y las vistas muestran lo deseado, además de buscar posibles vulnerabilidades.

También se realizará una prueba de rendimiento para simular miles de peticiones concurrentes a la API para evaluar la respuesta de los servidores.

Implementación:

El despliegue de la aplicación en los servidores de AWS se hará a través de SSH conectándolo al github del proyecto para el correcto despliegue del sistema.

Utilizando GitHub Actions para una entrega continua del desarrollo, al momento en el que se hace merge en la rama main después de haber validado las pruebas y despliegue de la nueva versión en producción.

Al momento de acceder por primera vez los usuarios tendrán pequeños mensajes que los guíen a través de la aplicación con pasos sencillos para buscar, canciones o artistas y crear playlist.

Mantenimiento:

Debe existir un monitoreo activo en producción para detectar las fallas críticas para abordarse de manera inmediata con corrección.

Monitoreo de la deuda técnica para parchear vulnerabilidades existentes y mejorar funciones con respuesta lenta y así optimizar de manera adecuada las funciones, el sistema de colas, las consultas con los índices a la base de datos conforme el volumen de datos incrementa.

Además de implementar un sistema de reportes para que los usuarios puedan levantar un ticket y este automáticamente añada sus logs para detectar la posible causa del problema.

Análisis y Reflexión:

1. ¿Qué etapa del SDLC consideras más importante y por qué?

La arquitectura y diseño del sistema. Un mal diseño estructural puede causar problemas en etapas más avanzadas del sistema. Aunque el proyecto se planea de la mejor manera o se definan todos los roles posibles, la estructura del sistema puede causar retrasos al momento de comenzar a desarrollar y sobre todo si es necesario escalar el sistema más adelante.

2. ¿Qué riesgos existirían si se omite una etapa?

El equipo podría entregar un sistema que funciona pero siempre habrán fallos, retrasos y desorientación al momento de desarrollarlo. Dando como resultado algo que pareciera funcionar pero que fallará al poco tiempo.

3. ¿Por qué el mantenimiento es crítico en sistemas modernos?

Porque además de reparar los errores que surjan, muchas tecnologías se actualizan constantemente, por tanto siempre se debe estar atento por cualquier actualización que requiera intervención en el sistema, como una dependencia que se actualiza o que haya sufrido alguna vulnerabilidad de la cual haya que encargarse.

4. ¿Cómo ayuda el SDLC a mejorar calidad y organización?

Se vuelve un proceso más estructurado y con metodología que determina la manera en la que el equipo de desarrollo tiene que trabajar, facilitando así la manera en la que se desarrolla el sistema ya que todos trabajan con la misma estructura de principio a fin.

Conclusión

En conclusión, el análisis del Ciclo de Vida del Software aplicado a una plataforma de la magnitud de Spotify demuestra que la construcción de sistemas informáticos exitosos va mucho más allá de la simple escritura de código. Como se evidenció a lo largo del documento, fases como el diseño de la arquitectura de la base de datos y la planificación de la infraestructura técnica actúan como los cimientos que evitan el colapso del sistema frente a escenarios de alta concurrencia. Omitir o subestimar cualquiera de las etapas del SDLC resulta en vulnerabilidades operativas, incremento de la deuda técnica y, en última instancia, en el fracaso del producto. Asimismo, se destaca que el ciclo de vida no termina con el despliegue; el mantenimiento constante, el monitoreo activo y la actualización de dependencias son actividades críticas para garantizar que el software se adapte y sobreviva en un entorno tecnológico que evoluciona rápidamente. En definitiva, la adopción rigurosa del SDLC es lo que permite a los equipos de desarrollo entregar soluciones robustas, escalables y de alta calidad.